

KeyQuorum Lab

Browser GUI Manual

Bailey Forbes

A guide to KeyQuorum Lab's own browser interface: its header and tabs, and the guided, gated tutorials built into the page. For the `keyquorum` / `keyquorum-device` command-line tool the lab runs underneath, see the separate KeyQuorum manual.

Contents

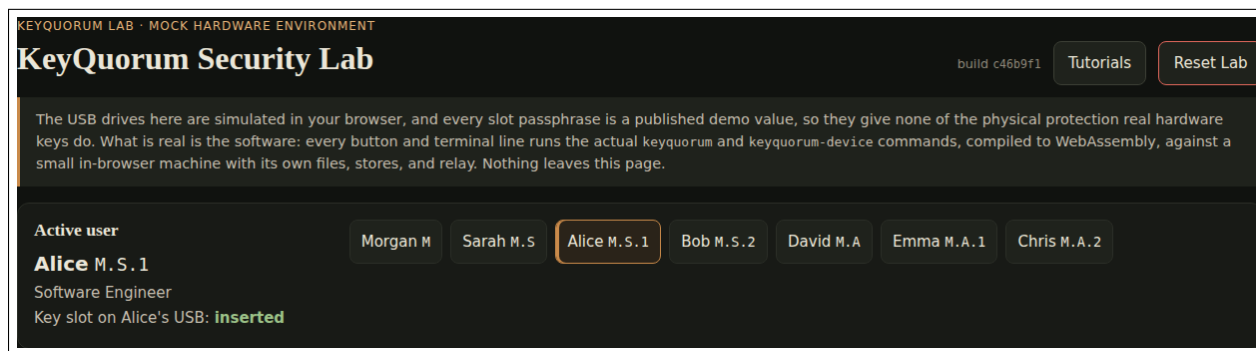
1	Overview	1
1.1	Identity and drives	1
2	The tabs	1
2.1	Organization key tree	1
2.2	Mock USB devices	2
2.3	File Explorer	3
2.4	Inbox and sent	3
2.5	Security & devices	4
2.6	Activity and access trace	4
2.7	Terminal	4
3	Guided tutorials	5
3.1	How a step works	5
3.2	The modules	5
4	Where the lab lives	6

1 Overview

KeyQuorum Lab runs the KeyQuorum crate in a browser as a small sandboxed machine, compiled to WebAssembly. It has its own files, a SQLite store per person, mock USB drives, and a mailbox relay answering in the page. Every button and every terminal line runs the real `keyquorum` and `keyquorum-device` commands — there is no separate “demo mode” standing in for them.

Note

This manual documents the lab’s own GUI: its header, tabs, and guided tutorials. It does not re-document `keyquorum` / `keyquorum-device` themselves — that is the separate *KeyQuorum* manual, which the lab runs underneath its interface unchanged. Where a control here is a thin wrapper over a CLI concept (custody, quorum, private bridges, authenticated updates, the mailbox relay), this manual names that concept without re-explaining it.



The lab header: build commit, the Tutorials and Reset Lab buttons, the disclaimer, and the Active user bar.

The mock USB drives are simulated entirely in the browser, and every slot passphrase is a published demo value, so they give none of the physical protection real hardware keys do. What is real is the software underneath: every action, in every tab, is one of the same commands documented in the KeyQuorum manual.

1.1 Identity and drives

The **Active user** bar shows who you are acting as right now, and which mock USB drive their personal key slot lives on. Clicking another name’s chip switches to them, the way sitting down at a different desk would. Most actions need that person’s drive *inserted* first — a mock drive behaves like a real one: its slots are only usable while it is connected, and moving a slot onto someone else’s drive needs both drives inserted at once, matching the physical requirement of moving a token between two real USB drives.

2 The tabs

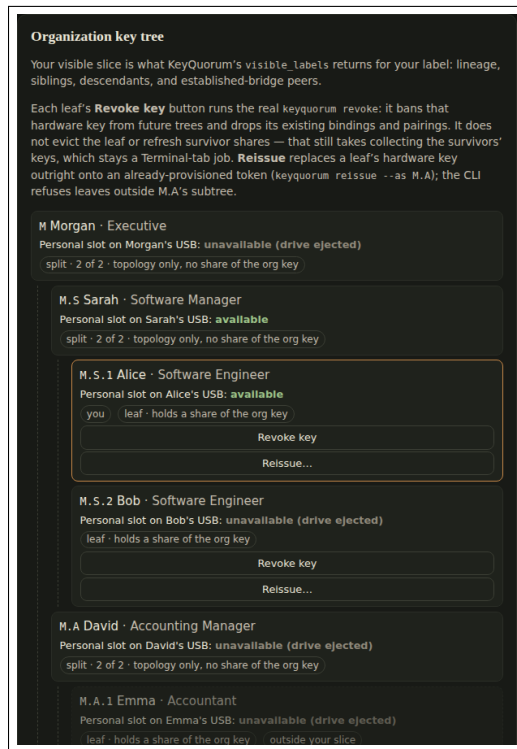
At desktop width every tab’s panel is visible at once, stacked on one scrolling page; the tab row itself only appears at phone width, where it switches which single panel is shown. What follows is every panel, in the order they appear on the page.

2.1 Organization key tree

The org-wide split tree, drawn as the dotted-label hierarchy it is: M, M.S, M.S.1, and so on. You only ever see your own slice — your lineage, your descendants, siblings, and established-bridge peers. A leaf tagged *required* contributed a share the last time you unlocked a file; *satisfied* means that share was actually presented.

- **Revoke key** bans a leaf’s hardware key outright; it drops that leaf’s bindings and pairings but does not evict it or refresh survivor shares.
- **Reissue...** replaces a leaf’s hardware key onto an already-provisioned replacement token.
- **Bridges** whitelists and establishes cross-branch links, the same two-step allow-then-add process the CLI uses.

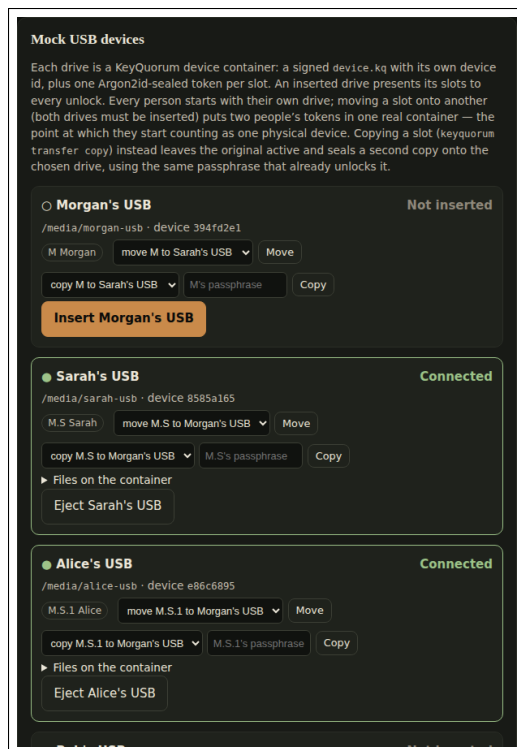
- **Tree restructure** walks through proposing a new public generation and collecting the parent's countersignature when one is required.



The organization key tree, showing the dotted-label hierarchy, per-leaf slot status, and the Revoke/Reissue controls.

2.2 Mock USB devices

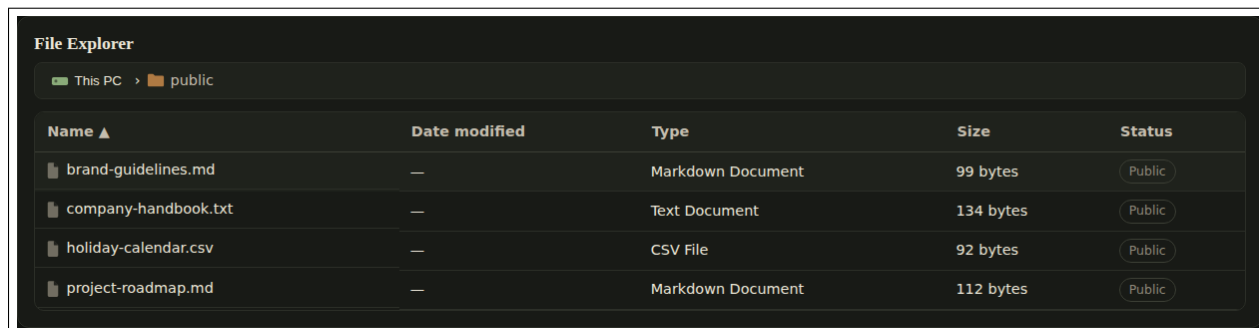
Each card is one device container: a signed device descriptor plus one sealed token per slot. **Insert** / **Eject** plug a drive in or pull it out. **Move** asks for a destination drive (both drives must be inserted) and relocates a slot's keypair onto it — the point where two people's tokens land in one physical container and start counting as a single device toward any multi-device quorum. **Copy** instead leaves the original slot active and seals a second copy elsewhere, using the same passphrase that already unlocks it.



Mock USB devices: connected state, the slots each drive carries, and the Move / Copy controls.

2.3 File Explorer

A Windows-Explorer-style view onto the active user's files, grouped into folders. Double-clicking a file (or selecting it and pressing **Open**) unlocks and views it — a quorum file prompts for the shares it needs, a password file for its password. **Send...** seals a copy to another label's key and drops it in their inbox; **Sign** is only offered on public or received plaintext; **Properties** shows the file's protection, size, and timestamps. Right-clicking a row offers the same actions. A file with an expiry stamp shows a status pill that flips to *expired* on its own while the tab is open.



The File Explorer, showing the public folder's contents and their status pills.

2.4 Inbox and sent

Letters are sealed to a recipient's key and routed by the mailbox relay without it ever reading them — sender and contents stay hidden until **Receive** unseals a letter with your own slot. **Reject** answers with a signed refusal instead. The Sent view tracks whether a delivery is still awaiting the recipient's signed acknowledgement, and **Check relay for acknowledgements** pulls any that are waiting.

Inbox and sent

Inbox (0) **Sent (0)**

Letters keyquorum relay pull fetched into ~/mail. The relay only knows the recipient key, so sender and file stay hidden until keyquorum deliver open unseals a letter with your slot.

No letters sealed to you.

Inbox and sent, in the empty seeded state.

2.5 Security & devices

Everything that is not the hardware-key quorum tree or a file: locking a note with a password (and optionally a PIN), exporting a file or credential as a portable sealed bundle, minting a time-limited share link, provisioning a fresh slot on an inserted drive and registering it as a new leaf, signing or verifying with a private sign bridge you belong to, and, further down, read-only device logs and relay counters for the in-process mailbox this lab runs against.

Security & devices

Password-locked files
Protect a note with a password only you choose (keyquorum access password), with an optional 4-digit PIN.

File name
my-note.txt

Contents
Whatever you want protected with a password

Your own lock password

☐ Also require a 4-digit PIN

Lock with my password

Your password-locked files
None yet.

Share links
Time-limited, revocable links (keyquorum share create-file). A bearer token authorizes redemption, not identity — anyone given the token can use it below.
None yet.

Exported bundles
Portable kqx8 bundles you sealed to another lab user's public key (keyquorum export file). No import step exists yet, so this only shows the sealed bytes.
None yet.

Create a new key
Provision a fresh slot on an inserted drive, sealed under a passphrase you choose.

Drive
Sarah's USB

New slot label
e.g. M.F

Your own passphrase

Security & devices: the password-lock form, exported bundles, share links, and key/leaf provisioning.

2.6 Activity and access trace

Every action taken leaves a trace here: which checks ran, in order, and whether each one passed. The **Equivalent operation** line under the latest trace is the literal command line that action ran — the same one that could be typed in the Terminal tab. The full log below keeps every action for the session, not just the last one.

2.7 Terminal

A real shell on the same lab machine. Every button on every other tab runs a command here under the hood; typing it directly works identically, including commands no button exposes yet.

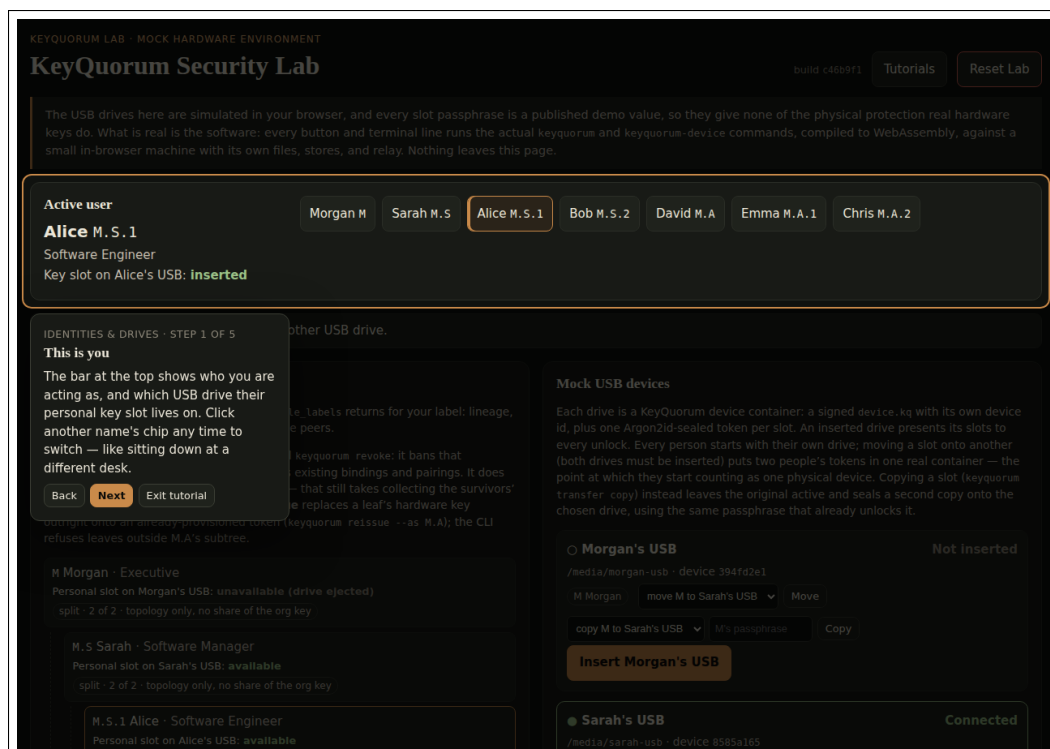
```
keyquorum-device list /media/alice-usb
bridge list 1
usb insert david
```

```
su david
```

`usb insert` / `usb eject` and `su` are lab-only shell verbs for drive and identity control; every other line is an ordinary `keyquorum` / `keyquorum-device` command.

3 Guided tutorials

The **Tutorials** button in the header opens a picker of short, self-contained modules. Each covers one part of the lab; picking one is not a commitment to the others, and they can be taken in any order.



A tutorial step in progress: the Active user bar is spotlighted, and the tooltip names the step and what to do.

3.1 How a step works

Each step spotlights the part of the page it is about and shows a tooltip with an explanation. A step comes in one of two kinds:

- An **informational** step just explains something; a **Next** button advances it.
- A **gated** step (marked “Waiting for you to try it...”) names a real action to perform — insert a drive, open a file, switch identity, send a letter — and does not advance until that action actually happens in the lab. Completion is checked against the lab’s own state (the current snapshot, or the most recent entry in the Activity log), not against a click on the tooltip itself, so there is no way to fake having done it. Once satisfied, the step marks itself done and advances on its own a moment later. **Skip this step** is always available if you would rather move on without doing it.

A step can also relocate its target as the page changes under it — for example, tracking a folder tile in the File Explorer until it is opened, then tracking the file row that takes its place — so one step can walk you through a short sequence rather than only ever pointing at one fixed element.

3.2 The modules

- **Identities & drives** — who you’re acting as, bringing a mock USB drive online, reading the organization key tree’s own slice, and switching identity.

- **Files & unlocking** — opening a public file, then trying a quorum-protected one to see what the access trace looks like when a single identity does not meet its threshold.
- **Mailbox: sending & receiving** — sealing a file to another label, switching to that identity, and receiving it from the inbox.

Finishing a module's last step opens a short completion notice; from there, or from **Exit tutorial** at any point, the **Tutorials** button reopens the picker for another module.

4 Where the lab lives

KeyQuorum Lab is published from `lab/` to GitHub Pages and embedded by `bailey-forbes.com`. A GitHub Actions workflow tests, builds, and publishes it from `main` on every change. The lab is a sandboxed machine, not a second implementation of KeyQuorum: it must never reimplement quorum, custody, approval, visibility, bridge, or delivery rules, or add a gate of its own in front of them — if the CLI would allow something on a real machine, the lab allows it too.